FAZE CLAN MOVEMENTS

Group 28

Project Phase Three

Physical Database Design

Group Members:

[50991299 – MM Mansoor] – (Group Leader)
[50657305 – MF Shootariya] – (Member)
[46289186 – U Kajee] – (Member)
[43855695 – L Kaka] – (Member)
[53932595 – M Ahmed] – (Member)
[46025596 – Z Cassim] – (Member)
[46664734 – U Bada] – (Member)
[53565509 – F Jhetam] – (Member)

Phase 3: Physical Design

Database Design

This phase implements the physical database design for Faze Clan Movements in Oracle SQL Developer. The final script creates the database tables, primary keys, foreign keys, check constraints, indexes, views, sample data, and demonstration queries required for Phase 3. The structure follows the Phase 2 logical design and is kept in third normal form by storing each subject area in its own table and using foreign keys to represent relationships.

Database Objects

The database objects consist of tables, constraints, indexes, views, data loading statements, and one trigger-based audit feature. All SQL in this report matches the final working script used in Oracle SQL Developer.

Removal of Existing Objects

The script first removes existing views, trigger, sequence, and tables in a safe order. Child objects are dropped before parent objects to avoid foreign key dependency errors. The ROUTE table is also dropped, because earlier errors occurred when it remained in the schema and Oracle reported that the object name was already in use.

```
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_customer_statement'; EXCEPTION WHEN OTHERS THEN NULL;
END;
/
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_active_shipments'; EXCEPTION WHEN OTHERS THEN NULL;
END;
/
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_fleet_status'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP VIEW vw_monthly_revenue'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TRIGGER trg_payment_audit'; EXCEPTION WHEN OTHERS THEN NULL;
END;
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE seq_payment_audit'; EXCEPTION WHEN OTHERS THEN NULL;
END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE payment_audit CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS
THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE payment CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE invoice CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE cargo_item CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS
THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE shipment CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE dispatch CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE truck_driver CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS
THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE maintenance_record CASCADE CONSTRAINTS'; EXCEPTION WHEN
OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE inspection_record CASCADE CONSTRAINTS'; EXCEPTION WHEN
OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE fuel_record CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS
THEN NULL; END;
/
```

```
BEGIN EXECUTE IMMEDIATE 'DROP TABLE truck CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE driver CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE route CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE vehicle_type CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS
THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE employee CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE department CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS
THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE customer_address CASCADE CONSTRAINTS'; EXCEPTION WHEN
OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE customer CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
NULL; END;
/
```

Creation of New Tables

The following tables are created with appropriate Oracle data types. Primary keys uniquely identify rows, foreign keys enforce relationships between tables, and check constraints restrict values to valid business values where applicable. The final version does not use GENERATED ALWAYS AS IDENTITY because the Oracle environment used for testing did not support it reliably.

Customer Table

Stores customer details. Email is unique and must follow a basic email pattern.

```
CREATE TABLE customer (
    customer_id     NUMBER(6) PRIMARY KEY,
    customer_name   VARCHAR2(80) NOT NULL,
    contact_number  VARCHAR2(20) NOT NULL,
    email           VARCHAR2(100) UNIQUE,
    CONSTRAINT chk_customer_email CHECK (email IS NULL OR email LIKE '%@%.%')
);
```

Customer Address Table

Stores multiple addresses for a customer and represents the multi-valued address attribute from the ER diagram.

```
CREATE TABLE customer_address (
    address_id    NUMBER(6) PRIMARY KEY,
    street        VARCHAR2(100) NOT NULL,
    city          VARCHAR2(50) NOT NULL,
    province      VARCHAR2(50) NOT NULL,
    postal_code   VARCHAR2(10) NOT NULL,
    customer_id   NUMBER(6) NOT NULL,
    CONSTRAINT fk_address_customer FOREIGN KEY (customer_id)
        REFERENCES customer(customer_id)
);
```

Department Table

Stores company departments used to classify employees.

```
CREATE TABLE department (
    department_id    NUMBER(4) PRIMARY KEY,
    department_name  VARCHAR2(60) NOT NULL UNIQUE
);
```

Employee Table

Stores employee details and links each employee to a department.

```
CREATE TABLE employee (
    employee_id     NUMBER(6) PRIMARY KEY,
    employee_fname  VARCHAR2(40) NOT NULL,
    employee_lname  VARCHAR2(40) NOT NULL,
    contact_number  VARCHAR2(20) NOT NULL,
    job_title       VARCHAR2(60) NOT NULL,
```

```
    department_id   NUMBER(4) NOT NULL,
    CONSTRAINT fk_employee_department FOREIGN KEY (department_id)
        REFERENCES department(department_id)
);
```

Driver Table

Stores driver-specific details and links each driver to an employee. Driver availability and licence validity are controlled by check constraints.

```
CREATE TABLE driver (
    driver_id            NUMBER(6) PRIMARY KEY,
    licence_number       VARCHAR2(30) NOT NULL UNIQUE,
    licence_expiry_date  DATE NOT NULL,
    availability_status  VARCHAR2(20) DEFAULT 'Available' NOT NULL,
    employee_id          NUMBER(6) NOT NULL UNIQUE,
    CONSTRAINT fk_driver_employee FOREIGN KEY (employee_id)
        REFERENCES employee(employee_id),
    CONSTRAINT chk_driver_availability CHECK
        (availability_status IN ('Available','Assigned','Unavailable','On Leave')),
    CONSTRAINT chk_driver_licence_valid CHECK
        (licence_expiry_date >= DATE '2026-01-01')
);
```

Vehicle Type Table

Stores the type/classification of each truck.

```
CREATE TABLE vehicle_type (
    vehicle_type_id  NUMBER(4) PRIMARY KEY,
    type_name        VARCHAR2(40) NOT NULL UNIQUE
);
```

Truck Table

Stores truck details, capacity, availability status, and vehicle type.

```
CREATE TABLE truck (
    truck_id             NUMBER(6) PRIMARY KEY,
    registration_number  VARCHAR2(20) NOT NULL UNIQUE,
    capacity             NUMBER(8,2) NOT NULL,
    availability_status  VARCHAR2(20) DEFAULT 'Available' NOT NULL,
    vehicle_type_id      NUMBER(4) NOT NULL,
    CONSTRAINT fk_truck_vehicle_type FOREIGN KEY (vehicle_type_id)
        REFERENCES vehicle_type(vehicle_type_id),
    CONSTRAINT chk_truck_capacity CHECK (capacity > 0),
    CONSTRAINT chk_truck_availability CHECK
        (availability_status IN ('Available','Assigned','Maintenance','Unavailable'))
);
```

Truck Driver Table

Bridge table resolving the many-to-many relationship between trucks and drivers.

```
CREATE TABLE truck_driver (
    truck_id    NUMBER(6) NOT NULL,
    driver_id   NUMBER(6) NOT NULL,
    uses        VARCHAR2(40) DEFAULT 'Delivery' NOT NULL,
    CONSTRAINT pk_truck_driver PRIMARY KEY (truck_id, driver_id),
    CONSTRAINT fk_td_truck FOREIGN KEY (truck_id)
        REFERENCES truck(truck_id),
    CONSTRAINT fk_td_driver FOREIGN KEY (driver_id)
        REFERENCES driver(driver_id)
);
```

Route Table

Stores route name, distance, and estimated delivery time. Distance and time must be greater than zero.

```
CREATE TABLE route (
    route_id                NUMBER(6) PRIMARY KEY,
    route_name              VARCHAR2(80) NOT NULL,
    distance                NUMBER(8,2) NOT NULL,
    estimated_delivery_time  NUMBER(5,2) NOT NULL,
    CONSTRAINT chk_route_distance CHECK (distance > 0),
    CONSTRAINT chk_route_time CHECK (estimated_delivery_time > 0)
);
```

Dispatch Table

Stores dispatch scheduling details and links each dispatch record to an employee.

```
CREATE TABLE dispatch (
    dispatch_id    NUMBER(6) PRIMARY KEY,
    dispatch_date  DATE NOT NULL,
    schedule_time  VARCHAR2(5) NOT NULL,
    employee_id    NUMBER(6) NOT NULL,
    CONSTRAINT fk_dispatch_employee FOREIGN KEY (employee_id)
        REFERENCES employee(employee_id)
);
```

Shipment Table

Central operational table linking customers, trucks, drivers, dispatch records, and routes. Shipment status is restricted to the accepted delivery lifecycle values.

```
CREATE TABLE shipment (
    shipment_id        NUMBER(6) PRIMARY KEY,
    shipment_date      DATE NOT NULL,
    pickup_location    VARCHAR2(120) NOT NULL,
    delivery_location  VARCHAR2(120) NOT NULL,
    shipment_status    VARCHAR2(20) DEFAULT 'Scheduled' NOT NULL,
    customer_id        NUMBER(6) NOT NULL,
    truck_id           NUMBER(6) NOT NULL,
    driver_id          NUMBER(6) NOT NULL,
    dispatch_id        NUMBER(6) NOT NULL UNIQUE,
    route_id           NUMBER(6) NOT NULL,
    CONSTRAINT fk_shipment_customer FOREIGN KEY (customer_id)
        REFERENCES customer(customer_id),
    CONSTRAINT fk_shipment_truck FOREIGN KEY (truck_id)
        REFERENCES truck(truck_id),
    CONSTRAINT fk_shipment_driver FOREIGN KEY (driver_id)
        REFERENCES driver(driver_id),
    CONSTRAINT fk_shipment_dispatch FOREIGN KEY (dispatch_id)
        REFERENCES dispatch(dispatch_id),
    CONSTRAINT fk_shipment_route FOREIGN KEY (route_id)
        REFERENCES route(route_id),
    CONSTRAINT chk_shipment_status CHECK
        (shipment_status IN ('Scheduled','Collected','In Transit','Delivered','Cancelled'))
);
```

Cargo Item Table

Stores cargo items for shipments. Each cargo item belongs to one shipment and weight must be greater than zero.

```
CREATE TABLE cargo_item (
    cargo_id               NUMBER(6) PRIMARY KEY,
    description            VARCHAR2(100) NOT NULL,
    weight                 NUMBER(8,2) NOT NULL,
    handling_requirements  VARCHAR2(100),
    shipment_id            NUMBER(6) NOT NULL,
    CONSTRAINT fk_cargo_shipment FOREIGN KEY (shipment_id)
        REFERENCES shipment(shipment_id),
    CONSTRAINT chk_cargo_weight CHECK (weight > 0)
);
```

Invoice Table

Stores invoice records. Each invoice belongs to one shipment and one customer.

```
CREATE TABLE invoice (
    invoice_id    NUMBER(6) PRIMARY KEY,
    invoice_date  DATE NOT NULL,
    amount        NUMBER(10,2) NOT NULL,
    shipment_id   NUMBER(6) NOT NULL UNIQUE,
    customer_id   NUMBER(6) NOT NULL,
    CONSTRAINT fk_invoice_shipment FOREIGN KEY (shipment_id)
        REFERENCES shipment(shipment_id),
    CONSTRAINT fk_invoice_customer FOREIGN KEY (customer_id)
        REFERENCES customer(customer_id),
    CONSTRAINT chk_invoice_amount CHECK (amount > 0)
);
```

Payment Table

Stores payments made against invoices. Multiple payments may be linked to a single invoice.

```
CREATE TABLE payment (
    payment_id      NUMBER(6) PRIMARY KEY,
    invoice_id      NUMBER(6) NOT NULL,
```

```
    payment_date    DATE NOT NULL,
    amount_paid     NUMBER(10,2) NOT NULL,
    payment_method  VARCHAR2(20) NOT NULL,
    customer_id     NUMBER(6) NOT NULL,
    CONSTRAINT fk_payment_invoice FOREIGN KEY (invoice_id)
        REFERENCES invoice(invoice id),
    CONSTRAINT fk_payment_customer FOREIGN KEY (customer_id)
        REFERENCES customer(customer_id),
    CONSTRAINT chk_payment_amount CHECK (amount_paid > 0),
    CONSTRAINT chk_payment_method CHECK
        (payment method IN ('Cash','Card','EFT','Account'))
);
```

Fuel Record Table

Tracks fuel usage and fuel cost for each truck.

```
CREATE TABLE fuel_record (
    fuel_id      NUMBER(6) PRIMARY KEY,
    fuel_date    DATE NOT NULL,
    litres_used  NUMBER(8,2) NOT NULL,
    cost         NUMBER(10,2) NOT NULL,
    truck_id     NUMBER(6) NOT NULL,
    CONSTRAINT fk_fuel_truck FOREIGN KEY (truck_id)
        REFERENCES truck(truck id),
    CONSTRAINT chk_fuel_litres CHECK (litres_used > 0),
    CONSTRAINT chk_fuel_cost CHECK (cost >= 0)
);
```

Inspection Record Table

Stores truck inspection outcomes.

```
CREATE TABLE inspection_record (
    inspection_id       NUMBER(6) PRIMARY KEY,
    inspection_date     DATE NOT NULL,
    inspection_results  VARCHAR2(20) NOT NULL,
    truck_id            NUMBER(6) NOT NULL,
    CONSTRAINT fk_inspection_truck FOREIGN KEY (truck id)
        REFERENCES truck(truck_id),
    CONSTRAINT chk_inspection_result CHECK
        (inspection results IN ('Passed','Failed','Pending'))
);
```

Maintenance Record Table

Stores maintenance services and costs for trucks.

```
CREATE TABLE maintenance_record (
    maintenance_id    NUMBER(6) PRIMARY KEY,
    maintenance_date  DATE NOT NULL,
    maintenance_type  VARCHAR2(60) NOT NULL,
    cost              NUMBER(10,2) NOT NULL,
    truck_id          NUMBER(6) NOT NULL,
    CONSTRAINT fk_maintenance_truck FOREIGN KEY (truck_id)
        REFERENCES truck(truck id),
    CONSTRAINT chk_maintenance_cost CHECK (cost >= 0)
);
```

Extra Functionality: Payment Audit Trigger

A sequence, audit table, and trigger are used to automatically record each payment inserted into the PAYMENT table. This demonstrates extra functionality and provides a basic audit trail for finance records.

```
CREATE SEQUENCE seq_payment_audit
START WITH 1
INCREMENT BY 1;

CREATE TABLE payment_audit (
    audit_id       NUMBER PRIMARY KEY,
    payment_id     NUMBER(6),
    audit_message  VARCHAR2(150),
    audit_date     DATE DEFAULT SYSDATE
);

CREATE OR REPLACE TRIGGER trg_payment_audit
AFTER INSERT ON payment
```

```
FOR EACH ROW
BEGIN
    INSERT INTO payment audit(audit id, payment id, audit message, audit date)
    VALUES (
        seq_payment_audit.NEXTVAL,
        :NEW.payment id,
        'Payment captured for invoice ' || :NEW.invoice_id,
        SYSDATE
    );
END;
/
```

Creating Indexes

Indexes are created to improve the performance of common searches, joins, and reporting queries. These indexes support customer lookups, shipment filtering, payment-to-invoice joins, and fleet status reporting.

```
CREATE INDEX idx customer name ON customer(customer name);
CREATE INDEX idx_shipment_status ON shipment(shipment_status);
CREATE INDEX idx shipment customer ON shipment(customer id);
CREATE INDEX idx_payment_invoice ON payment(invoice_id);
CREATE INDEX idx_truck_status ON truck(availability_status);
```

Creating Views

Views simplify reporting by storing reusable SELECT statements. The views below support operations, finance, fleet monitoring, and revenue reporting.

Active Shipments View

Lists all shipments that have not yet been delivered.

```
CREATE OR REPLACE VIEW vw active shipments AS
SELECT
    s.shipment id,
    c.customer name,
    s.pickup location,
    s.delivery location,
    s.shipment_status,
    e.employee fname || ' ' || e.employee lname AS driver name,
    t.registration number,
    r.route name
FROM shipment s
JOIN customer c ON s.customer_id = c.customer_id
JOIN driver dr ON s.driver id = dr.driver id
JOIN employee e ON dr.employee_id = e.employee_id
JOIN truck t ON s.truck id = t.truck id
JOIN route r ON s.route id = r.route id
WHERE s.shipment_status <> 'Delivered';
```

Customer Statement View

Calculates the total paid and outstanding balance per invoice.

```
CREATE OR REPLACE VIEW vw customer statement AS
SELECT
    c.customer id,
    c.customer_name,
    i.invoice id,
    i.amount AS invoice amount,
    NVL(SUM(p.amount_paid), 0) AS total_paid,
    i.amount - NVL(SUM(p.amount paid), 0) AS outstanding balance
FROM customer c
JOIN invoice i ON c.customer id = i.customer id
LEFT JOIN payment p ON i.invoice id = p.invoice id
GROUP BY c.customer_id, c.customer_name, i.invoice_id, i.amount;
```

Fleet Status View

Shows truck capacity, type, status, and latest inspection date.

```
CREATE OR REPLACE VIEW vw_fleet_status AS
SELECT
    t.truck id,
    t.registration number,
    vt.type name,
    t.capacity,
    t.availability_status,
```

```
    MAX(ir.inspection_date) AS latest_inspection_date
FROM truck t
JOIN vehicle_type vt ON t.vehicle_type_id = vt.vehicle_type_id
LEFT JOIN inspection_record ir ON t.truck_id = ir.truck_id
GROUP BY t.truck_id, t.registration_number, vt.type_name, t.capacity, t.availability_status;
```

Monthly Revenue View

Summarises invoices by month.

```
CREATE OR REPLACE VIEW vw_monthly_revenue AS
SELECT
    TRUNC(invoice_date, 'MM') AS revenue_month,
    COUNT(invoice_id) AS number_of_invoices,
    SUM(amount) AS total_invoiced
FROM invoice
GROUP BY TRUNC(invoice_date, 'MM');
```

Populating Tables

The following INSERT statements load sample operational data into the database. The data is sufficient to demonstrate customer records, addresses, employees, drivers, trucks, routes, dispatching, shipments, cargo, invoicing, payments, inspections, fuel usage, and maintenance records.

```
INSERT INTO customer VALUES (1, 'Apex Retailers', '0115551001', 'orders@apex.co.za');
INSERT INTO customer VALUES (2, 'BlueStone Hardware', '0125552002',
'logistics@bluestone.co.za');
INSERT INTO customer VALUES (3, 'CapeFresh Foods', '0215553003', 'dispatch@capefresh.co.za');
INSERT INTO customer VALUES (4, 'Durban Tech Supplies', '0315554004',
'info@durbantech.co.za');
INSERT INTO customer VALUES (5, 'EcoBuild Materials', '0105555005',
'accounts@ecobuild.co.za');

INSERT INTO customer_address VALUES (1, '10 Main Road', 'Johannesburg', 'Gauteng', '2001',
1);
INSERT INTO customer_address VALUES (2, '18 Market Street', 'Pretoria', 'Gauteng', '0002',
2);
INSERT INTO customer_address VALUES (3, '55 Harbour Road', 'Cape Town', 'Western Cape',
'8001', 3);
INSERT INTO customer_address VALUES (4, '22 Umgeni Road', 'Durban', 'KwaZulu-Natal', '4001',
4);
INSERT INTO customer_address VALUES (5, '7 Industrial Park', 'Bloemfontein', 'Free State',
'9301', 5);

INSERT INTO department VALUES (1, 'Operations and Dispatch');
INSERT INTO department VALUES (2, 'Fleet Management');
INSERT INTO department VALUES (3, 'Finance and Administration');
INSERT INTO department VALUES (4, 'IT and Database Management');

INSERT INTO employee VALUES (1, 'Amina', 'Patel', '0821110001', 'Dispatch Officer', 1);
INSERT INTO employee VALUES (2, 'Thabo', 'Mokoena', '0821110002', 'Fleet Manager', 2);
INSERT INTO employee VALUES (3, 'Sipho', 'Dlamini', '0821110003', 'Driver', 1);
INSERT INTO employee VALUES (4, 'Riya', 'Naidoo', '0821110004', 'Driver', 1);
INSERT INTO employee VALUES (5, 'Farhan', 'Khan', '0821110005', 'Finance Clerk', 3);
INSERT INTO employee VALUES (6, 'Lerato', 'Nkosi', '0821110006', 'Driver', 1);

INSERT INTO driver VALUES (1, 'GP-DL-1001', DATE '2028-05-20', 'Assigned', 3);
INSERT INTO driver VALUES (2, 'KZN-DL-2002', DATE '2029-07-15', 'Assigned', 4);
INSERT INTO driver VALUES (3, 'WC-DL-3003', DATE '2027-11-30', 'Available', 6);

INSERT INTO vehicle_type VALUES (1, 'Light Truck');
INSERT INTO vehicle_type VALUES (2, 'Heavy Truck');
INSERT INTO vehicle_type VALUES (3, 'Refrigerated Truck');

INSERT INTO truck VALUES (1, 'GP123456', 3500, 'Assigned', 1);
INSERT INTO truck VALUES (2, 'KZN654321', 9000, 'Assigned', 2);
INSERT INTO truck VALUES (3, 'WC777888', 5000, 'Available', 3);
INSERT INTO truck VALUES (4, 'FS444222', 8000, 'Maintenance', 2);

INSERT INTO truck_driver VALUES (1, 1, 'Local deliveries');
INSERT INTO truck_driver VALUES (2, 2, 'Long distance deliveries');
```

```
INSERT INTO truck driver VALUES (3, 3, 'Cold-chain deliveries');
INSERT INTO truck_driver VALUES (2, 1, 'Backup driver');

INSERT INTO route VALUES (1, 'Johannesburg to Pretoria', 65, 1.50);
INSERT INTO route VALUES (2, 'Johannesburg to Durban', 570, 7.50);
INSERT INTO route VALUES (3, 'Cape Town to Bloemfontein', 1000, 12.00);
INSERT INTO route VALUES (4, 'Durban to Johannesburg', 570, 7.25);

INSERT INTO dispatch VALUES (1, DATE '2026-04-01', '08:00', 1);
INSERT INTO dispatch VALUES (2, DATE '2026-04-02', '09:30', 1);
INSERT INTO dispatch VALUES (3, DATE '2026-04-03', '07:45', 1);
INSERT INTO dispatch VALUES (4, DATE '2026-04-04', '10:15', 1);

INSERT INTO shipment VALUES (1, DATE '2026-04-01', 'Johannesburg Depot', 'Pretoria
Warehouse', 'Delivered', 1, 1, 1, 1);
INSERT INTO shipment VALUES (2, DATE '2026-04-02', 'Johannesburg Depot', 'Durban Tech
Supplies', 'In Transit', 4, 2, 2, 2, 2);
INSERT INTO shipment VALUES (3, DATE '2026-04-03', 'Cape Town Depot', 'Bloemfontein Site',
'Scheduled', 5, 3, 3, 3, 3);
INSERT INTO shipment VALUES (4, DATE '2026-04-04', 'Durban Depot', 'Johannesburg Hub',
'Collected', 2, 2, 1, 4, 4);

INSERT INTO cargo item VALUES (1, 'Electronics boxes', 450, 'Fragile', 1);
INSERT INTO cargo_item VALUES (2, 'Hardware pallets', 1800, 'Forklift required', 2);
INSERT INTO cargo item VALUES (3, 'Fresh produce crates', 1200, 'Refrigerated', 3);
INSERT INTO cargo_item VALUES (4, 'Building cement bags', 3000, 'Keep dry', 4);
INSERT INTO cargo item VALUES (5, 'Office furniture', 700, 'Handle with care', 1);

INSERT INTO invoice VALUES (1, DATE '2026-04-01', 3500.00, 1, 1);
INSERT INTO invoice VALUES (2, DATE '2026-04-02', 12500.00, 2, 4);
INSERT INTO invoice VALUES (3, DATE '2026-04-03', 21000.00, 3, 5);
INSERT INTO invoice VALUES (4, DATE '2026-04-04', 9800.00, 4, 2);

INSERT INTO payment VALUES (1, 1, DATE '2026-04-01', 3500.00, 'EFT', 1);
INSERT INTO payment VALUES (2, 2, DATE '2026-04-03', 6000.00, 'Card', 4);
INSERT INTO payment VALUES (3, 2, DATE '2026-04-05', 3000.00, 'EFT', 4);
INSERT INTO payment VALUES (4, 4, DATE '2026-04-05', 5000.00, 'Cash', 2);

INSERT INTO fuel record VALUES (1, DATE '2026-04-01', 45.50, 1150.00, 1);
INSERT INTO fuel record VALUES (2, DATE '2026-04-02', 180.00, 4550.00, 2);
INSERT INTO fuel_record VALUES (3, DATE '2026-04-03', 120.00, 3000.00, 3);

INSERT INTO inspection_record VALUES (1, DATE '2026-03-28', 'Passed', 1);
INSERT INTO inspection record VALUES (2, DATE '2026-03-29', 'Passed', 2);
INSERT INTO inspection_record VALUES (3, DATE '2026-04-01', 'Passed', 3);
INSERT INTO inspection_record VALUES (4, DATE '2026-04-01', 'Failed', 4);

INSERT INTO maintenance record VALUES (1, DATE '2026-03-20', 'Tyre replacement', 4200.00, 1);
INSERT INTO maintenance_record VALUES (2, DATE '2026-03-25', 'Brake service', 6500.00, 2);
INSERT INTO maintenance record VALUES (3, DATE '2026-04-02', 'Engine diagnostics', 2800.00,
4);

COMMIT;
```

SQL Select Statements

The following queries are based on the information requirements of Faze Clan Movements and demonstrate the required SQL concepts in the Phase 3 marking guide.

Statement 1: Limitation of Rows and Columns

Retrieves only three columns from the SHIPMENT table and limits the output to the first three records.

```
SELECT shipment_id, shipment_status, delivery_location
FROM shipment
WHERE ROWNUM <= 3;
```

Statement 2: Sorting

Sorts shipment records from newest to oldest.

```
SELECT shipment_id, shipment_date, shipment_status
FROM shipment
ORDER BY shipment_date DESC;
```

Statement 3: LIKE, AND and OR

Uses LIKE with AND/OR conditions to search customer data.

```
SELECT customer_id, customer_name, email
FROM customer
WHERE (customer_name LIKE '%Tech%' OR customer_name LIKE '%Hardware%')
AND email LIKE '%.za';
```

Statement 4: Character Functions

Uses UPPER and LOWER to demonstrate character functions while searching customer names.

```
SELECT
    customer_id,
    UPPER(customer_name) AS customer_name_upper,
    LOWER(email) AS email_lower
FROM customer
WHERE UPPER(customer_name) LIKE UPPER('%Tech%');
```

Statement 5: ROUND Function

Calculates average kilometres per hour for each route and rounds the value to two decimals.

```
SELECT
    route_id,
    route_name,
    distance,
    ROUND(distance / estimated_delivery_time, 2) AS average_km_per_hour
FROM route;
```

Statement 6: Date Functions

Uses ADD_MONTHS and SYSDATE calculations to analyse shipment dates.

```
SELECT
    shipment_id,
    shipment_date,
    ADD_MONTHS(shipment_date, 1) AS follow_up_date,
    TRUNC(SYSDATE) - shipment_date AS days_since_shipment
FROM shipment;
```

Statement 7: Aggregate Functions

Summarises total shipments and invoice amounts using COUNT, SUM, AVG, and MAX.

```
SELECT
    COUNT(*) AS total_shipments,
    SUM(amount) AS total_invoice_amount,
    AVG(amount) AS average_invoice_amount,
    MAX(amount) AS largest_invoice
FROM invoice;
```

Statement 8: GROUP BY and HAVING

Groups shipments and invoices by customer and filters customers with more than R5 000 invoiced.

```
SELECT
    c.customer_name,
    COUNT(s.shipment_id) AS number_of_shipments,
    SUM(i.amount) AS total_invoiced
FROM customer c
JOIN shipment s ON c.customer_id = s.customer_id
JOIN invoice i ON s.shipment_id = i.shipment_id
GROUP BY c.customer_name
HAVING SUM(i.amount) > 5000;
```

Statement 9: Joins

Combines shipment, customer, truck, and route data into a single operational report.

```
SELECT
    s.shipment_id,
    c.customer_name,
    t.registration_number,
    r.route_name,
    s.shipment_status
FROM shipment s
JOIN customer c ON s.customer_id = c.customer_id
JOIN truck t ON s.truck_id = t.truck_id
JOIN route r ON s.route_id = r.route_id;
```

Statement 10: Sub-query

Finds invoices with amounts greater than the average invoice amount.

```
SELECT invoice_id, amount, customer_id
FROM invoice
WHERE amount > (SELECT AVG(amount) FROM invoice);
```

Statement 11: Outstanding Balances

Uses the customer statement view to list invoices where balances remain outstanding.

```
SELECT *
FROM vw customer statement
WHERE outstanding_balance > 0
ORDER BY outstanding_balance DESC;
```

Statement 12: Active Deliveries

Uses the active shipments view to list shipments that have not yet been delivered.

```
SELECT *
FROM vw active shipments
ORDER BY shipment_status, delivery_location;
```

Statement 13: Fleet Maintenance Cost

Shows total maintenance cost per truck.

```
SELECT
    t.registration_number,
    SUM(m.cost) AS total maintenance cost
FROM truck t
JOIN maintenance_record m ON t.truck_id = m.truck_id
GROUP BY t.registration number
ORDER BY total_maintenance_cost DESC;
```

Statement 14: Payment Audit

Displays the payment audit records automatically inserted by the trigger.

```
SELECT *
FROM payment_audit;
```

Required SQL Concepts Covered

- Limitation of rows and columns: Statement 1
- Sorting: Statements 2, 11, 12 and 13
- LIKE, AND and OR: Statement 3
- Character functions: Statement 4
- ROUND and TRUNC/date calculations: Statements 5 and 6
- Date functions: Statement 6
- Aggregate functions: Statements 7, 8 and 13
- GROUP BY and HAVING: Statement 8
- Joins: Statements 8, 9 and 13
- Sub-queries: Statement 10
- Views: Statements 11 and 12
- Extra functionality: Payment audit trigger and PAYMENT_AUDIT table

Conclusion

The final SQL script successfully creates all required database objects, loads sample data, creates views and indexes, and executes the required demonstration queries. The output confirms that the tables, views, sequence, trigger, indexes, inserts, and queries run successfully in Oracle SQL Developer.